Sujet de TP 2 : Interface Capteur & STM32

La carte de développement **NUCLEO-N657X0** est basée sur un microcontrôleur ARM Cortex-M33 cadencé jusqu'à 160 MHz, avec une mémoire RAM de 320 ko et une mémoire Flash de 512 Ko. Elle est illustrée sur la figure 1. Elle offre des interfaces variées et dispose de ressources hardware avancé, comme un accélérateur cryptographique et neuromorphic processing unit (NPU) type Neural-ART (600 GOPS, 1 GHz), optimisé pour l'inférence de modèles CNN. La carte capteurs **X-NUCLEO-IKS01A3** regroupe plusieurs capteurs MEMS permettant de mesurer différentes grandeurs physiques telles que l'accélération, le champ magnétique, la température, l'humidité ou encore la pression atmosphérique. Ces capteurs communiquent avec le microcontrôleur via un bus I2C, largement utilisé dans les systèmes embarqués pour connecter des périphériques à faible débit. Elle est illustrée sur la figure 2.



Fig. 1. Carte de développement **NUCLEO-N657X0**

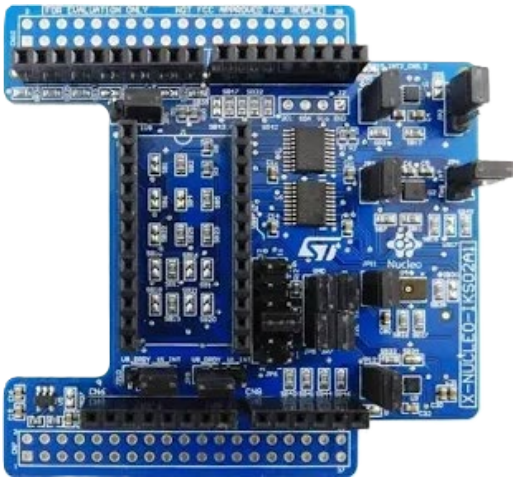


Fig. 2. Carte de développement **X-NUCLEO-IKS01A3**

1. LED Blink sur NUCLEO-STM32N657

Utiliser le logicielle STM32 Cube IDE pour programmer cette carte avec le tutoriel Blink LED qui est disponible sur : Wiki.

a. Allumer les trois LEDs (rouge, vert, bleu).
b. Implémenter un chenillard avec ce trois LEDS et une temporalisation de 3 secondes.
c. Imprimer sur la console les états de fonctionnement de la carte : <début d'application>, <début de chenillard>, <ROUGE>, <VERT>,<BLEU>, <début de chenillard>, ... Pour démontrer l'utilisation des logs sur interface console.

2. Interface Capteurs

L'objectif de ce travail pratique est de comprendre comment intégrer des drivers matériels fournis par le constructeur, configurer une interface I²C sur le microcontrôleur et implémenter les fonctions bas niveau permettant la lecture et l'écriture dans les registres des capteurs. Les étudiants devront ensuite utiliser ces drivers pour acquérir des données issues des capteurs et les afficher via l'interface série du microcontrôleur. Ce travail permettra ainsi de se familiariser avec la programmation embarquée, la communication avec des périphériques externes et l'utilisation de bibliothèques logicielles fournies par les fabricants. Ils sont disponibles sur https://github.com/STMicroelectronics/STMems_Standard_C_drivers une bibliothèque de drivers pour les capteurs sur la carte d'interface capteur identifiés par :

- **LSM6DSO** : 3-axis accelerometer + 3-axis gyroscope
- **LIS2MDL** : 3-axis magnetometer
- **LIS2DW12** : 3-axis accelerometer
- **HTS221** : humidity and temperature sensor
- **LPS22HH** : pressure sensor
- **STTS751** : temperature sensor

a. Proposer une implémentation sur **NUCLEO-STM32N657** capable de lire les valeurs des trois capteurs. Imprimer les valeurs lues comme un log sur l'interface console.
b. Modifier l'utilisation des LEDS de l'exercice précèdent pour qu'ils deviennent un repère visuel de l'état de fonctionnement de la carte, soit
   - <ROUGE> - Occupé en train de lire les capteurs
   - <VERT> - Disponible et en attente d'une prochaine lecture

Étapes de réalisation (procédure recommandée)

1. **Récupération des drivers**
   - Cloner / télécharger STMems_Standard_C_drivers depuis le dépôt indiqué.
   - Dans le dossier du driver, repérer pour chaque capteur les fichiers xxx_reg.c et xxx_reg.h.
   - Copier les paires xxx_reg.c / xxx_reg.h nécessaires dans un nouveau sous-dossier de votre projet (ex. Drivers/STMems/).
   - Ajouter ce dossier au include path du projet.
2. **Étudier l'exemple**

- Le dossier example du dépôt contient des templates montrant comment appeler read_reg / write_reg et comment effectuer une lecture de capteur. Lisez-les pour comprendre l'API fournie par xxx_reg.c/h.

3. **Implémenter les fonctions bas-niveau**
   - Les fichiers xxx_reg.c appellent des fonctions bas-niveau read_reg / write_reg (non implémentées par ST). Vous devez les remplacer par des appels I²C HAL adaptés à votre MCU.
   - Exemple-type (à adapter à vos noms d'instances et à la gestion d'adresse 7/8 bits) :

```c
int32_t platform_read(void *handle, uint8_t reg, uint8_t *buf, uint16_t len) {
    // handle peut être un pointeur vers l'I2C handle (ex. &hi2c2)
    I2C_HandleTypeDef *hi2c = (I2C_HandleTypeDef*) handle;
    uint16_t dev_addr = /* adresse 7-bit du capteur << 1 si HAL demande 8-bit */;

    if (HAL_I2C_Mem_Read(hi2c, dev_addr, reg, I2C_MEMADD_SIZE_8BIT, buf, len, 1000) == HAL_OK)
        return 0;
    else
        return -1;
}
```

- Implémentez symétriquement platform_write avec HAL_I2C_Mem_Write.
- **Remarque importante** : vérifiez si les adresses données dans la doc sont 7-bit ; HAL attend souvent l'adresse 8-bit (7-bit << 1). Testez et documentez votre choix.

4. **Configurer le bus I²C (CubeMX)**
   - Créez/générez MX_I2C2_Init() (ou autre interface choisie).
   - Exemple de mapping fourni : PA12 -> I2C2_SCL, PA11 -> I2C2_SDA (adaptez si votre carte diffère).
   - Sélectionnez vitesse (standard/fast) en fonction du capteur et de votre câblage. CubeMX génère les fichiers d'initialisation.
5. **Activer la sortie flottante pour printf (optionnel mais utile)**
   - Si vous souhaitez afficher des float via sprintf / printf, activez l'option _printf_float dans les paramètres du compilateur (Project Settings > MCU Settings dans CubeIDE).
6. **Tester la communication**
   - Avant de lire les mesures, testez la présence des périphériques I²C :
   - Tenter une lecture d'un registre connu (ex. WHO_AM_I) ou implémenter une boucle de scan d'adresses I²C.
   - Si les lectures échouent : vérifier les pull-ups I²C, câblage, adresses, et initialisation HAL.
7. **Lire et afficher les grandeurs**
   - Utiliser les fonctions fournies par xxx_reg.c (ex. stts751_temperature_raw_get ou équivalent) pour obtenir les valeurs brutes, puis appliquer les conversions (fourni dans les drivers).

- Afficher via UART/printf : température (°C), humidité (%RH), pression (hPa), accélération (mg / m/s²), magnétisme (µT), etc.

3. Réseau Ethernet

La carte de développement **NUCLEO-N657X0** possède un connecteur RJ45 classique pour une connexion à un réseau locale. En revanche, les protocoles de communication exigent un système d'exploitation temps réelle adapté à la carte.

a. Proposer une implémentation du système d'exploitation FreeRTOS sur la carte **NUCLEO-N657X0**

b. Implémenter un LWIP (stack IP) avec DHCP + ICMP et utiliser le Socket BSD dans la configuration de l'API/

c. Exécuter un ping sur google.com

d. Modifier l'utilisation des LEDS de l'exercice précèdent pour qu'ils deviennent un repère visuel de l'état de fonctionnement de la carte, soit

- <BLEU> - Communication réseau

e. Imprimer sur la console les états de fonctionnement de la carte : <début d'application>, <Lecture Interface Capteurs>, <Communication Réseau>, <Lecture Interface Capteurs>, <Communication Réseau>, ... Pour démontrer l'utilisation des logs sur interface console.

Étapes de réalisation (procédure recommandée)

1. **Préparer l'environnement**
   - Vérifier la compatibilité de la carte cible (NUCLEO-F746ZQ ou STM32F746G-DISCO) avec la pile LWIP et FreeRTOS fournis.
   - Ouvrir CubeMX / CubeIDE, sélectionner la carte et générer un projet de base (clock, périphériques ethernet si présents).
2. **Choix de l'API FreeRTOS (CMSISv1 vs CMSISv2)**
   - Décider d'utiliser CMSISv1 ou CMSISv2. CMSISv2 est fonctionnel mais consomme plus de pile.
   - **Si vous choisissez CMSISv2** : augmenter la pile du thread qui reçoit les paquets (ethernetif_input) pour éviter des débordements.
   - Exemple d'ajustement à appliquer dans ethernetif.c (ou là où le thread est créé) :

```c
memset(&attributes, 0x0, sizeof(osThreadAttr_t));
attributes.name = "EthIf";
attributes.stack_size = 1024; // augmenter si débordement (ex. 2048)
attributes.priority = osPriorityRealtime;
osThreadNew(ethernetif_input, netif, &attributes);
```

   - *Conseil* : si vous observez encore des stack overflow après 1024, essayez 1536 ou 2048 et reliiez cela à l'utilisation mémoire totale du système (tasks, heap).
3. **Configuration LWIP (mémoire et logs)**
   - Ouvrir la configuration LWIP dans CubeMX et augmenter la taille mémoire globale :
     - MEM_SIZE → **5000 bytes** (valeur recommandée initiale). Ajuster si nécessaire selon la charge (buffers, sockets ouverts).

- Activer les logs pour diagnostiquer :
  - Dans le fichier lwipopts.h ajouter (dans la zone USER CODE) :

```c
/* USER CODE BEGIN 0 */
#define LWIP_DEBUG
/* USER CODE END 0 */
```

- Recompiler et vérifier que les messages LWIP apparaissent sur la console (UART). Sinon, vérifier la configuration printf / retargeting.

4. **Résoudre les problèmes de CRC sur les paquets**
   - Si les pings ICMP montrent des réponses mais avec CRC fail sur le replay, le calcul CRC matériel peut ne pas être correctement configuré. Vérifier dans CubeMX :
     - Dans **Middleware > LWIP** : activer **CRC by hardware**, désactiver les autres options de CRC.
     - Dans **Configuration MCU > CRC** (ou périphériques) : activer l'option **Computing > CRC** (activer le périphérique CRC dans CubeMX).
   - Re-générer, recompiler et retester le ping.
5. **Tests et validations**
   - Démarrer l'interface réseau et effectuer des tests simples :
     - ping depuis une machine sur le même réseau vers l'adresse IP de la carte.
     - Ouvrir une socket BSD (socket, bind, listen / connect) et échanger de petites données (ex. echo simple).
   - Observer les logs LWIP et FreeRTOS pour détecter erreurs (timeouts, mem errors, stack overflow).
   - En cas d'échec, isoler :
     - Vérifier la configuration PHY (autonegotiation / duplex / speed) si applicable.
     - Vérifier l'initialisation du netif (adresse IP, netmask, gateway).
     - Vérifier les callbacks d'interruption et que le cache/invalidations (si CPU avec cache) sont correctement gérés lors des DMA.
6. **Optimisations et robustesse**
   - Ajuster MEM_SIZE si vous ouvrez plusieurs sockets simultanément.
   - Ajuster tailles de piles des tasks réseau si vous avez de nombreuses connexions concurrentes.
   - Mettre en place une gestion des erreurs LWIP (callbacks d'allocation, logs) pour redémarrer proprement l'interface si besoin.